

DATA 260 Homework 1

Section 0. Personal Configuration and Domain

Configuration	Value
SID4	9486
PORT_BASE	8486
PREFIX	s9486
SEED	9486
VERIFY_SEED	269486
DOMAIN_ID	6
Assigned domain	Rental housing listings
Hardware	Apple Mac with M1 chip and 8 GB unified memory
Local model	qwen3:4b
Content-complete source commit	832703b
Final submission tag	hw1

The recommended qwen3:8b model was replaced with qwen3:4b because the available Mac has 8 GB of unified memory. The smaller model is practical for repeated local runs and is documented as the permitted hardware-based substitution.

Reproducible commands

```
python3 -m http.server 8486 --directory app
docker build -t data260-hw1:s9486 .
docker run --name data260-hw1-s9486 -p 8486:80 data260-hw1:s9486
docker stop data260-hw1-s9486
docker rm data260-hw1-s9486
```

Section 1. Part 1 - Local Validation

The browser enforces the required description rule, and the JavaScript console records the successful JSON workflow.

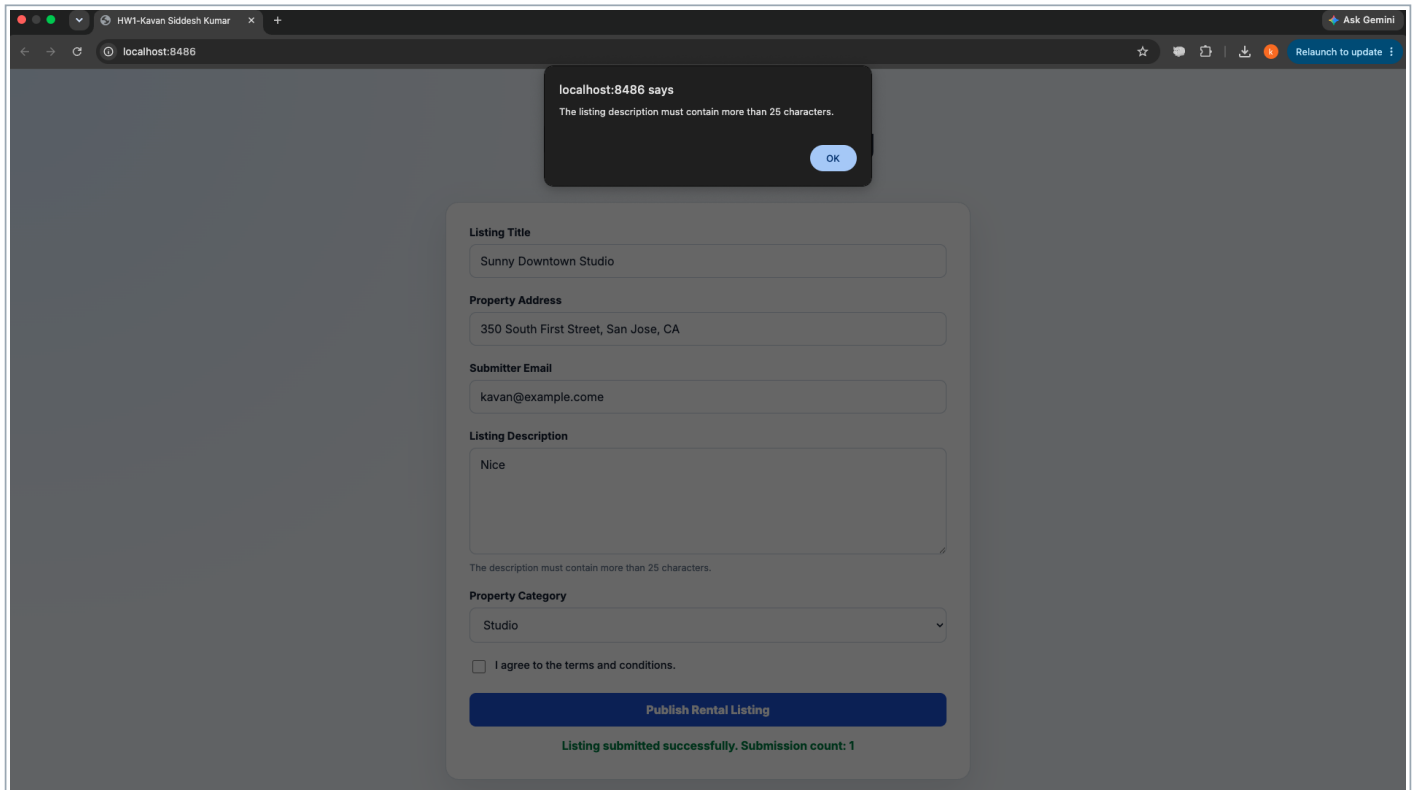


Figure 1. Short-description validation alert on localhost:8486

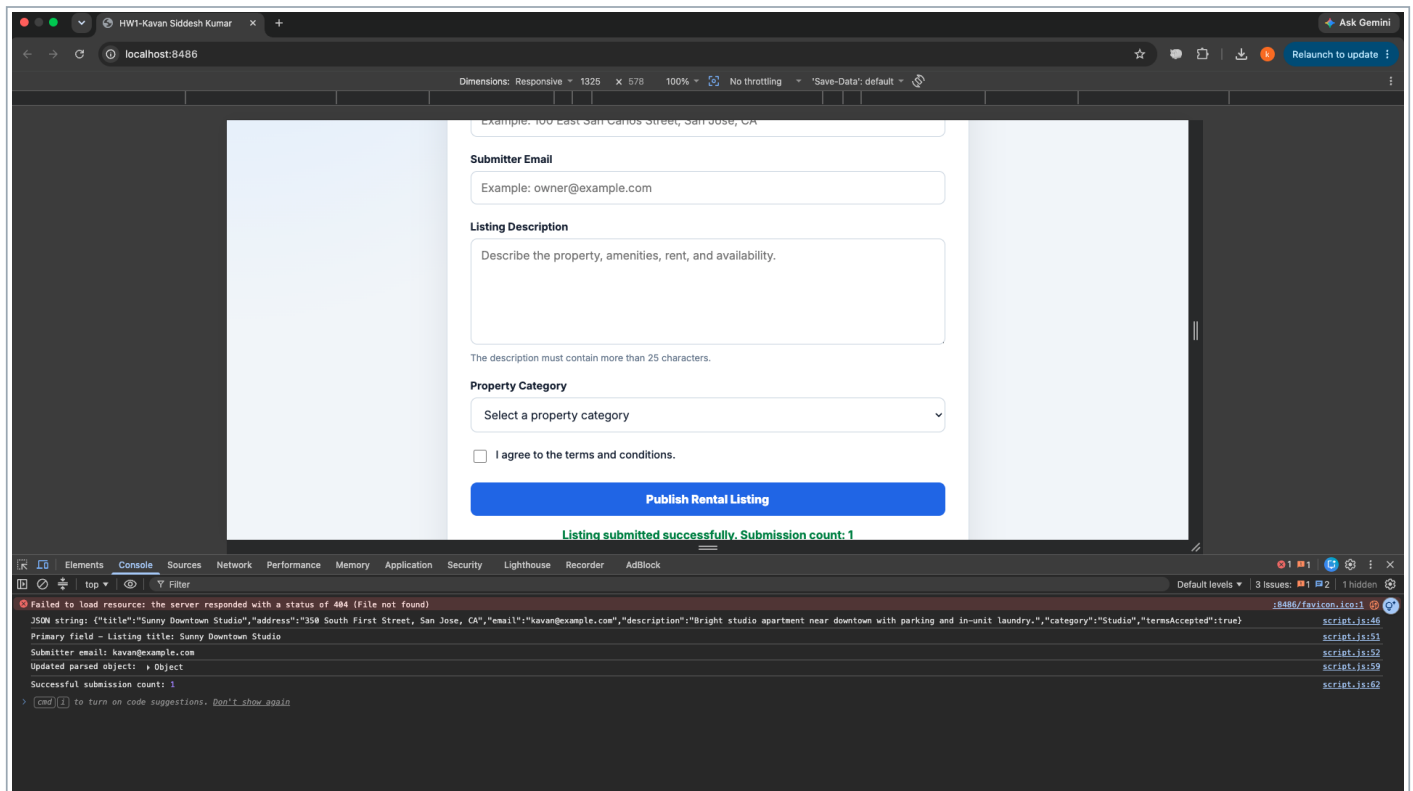


Figure 2. Console evidence: JSON, destructuring, spread update, and submission count

Section 1. Part 1 - Form Behavior

These screenshots show the required terms validation and the successful form-submission state.

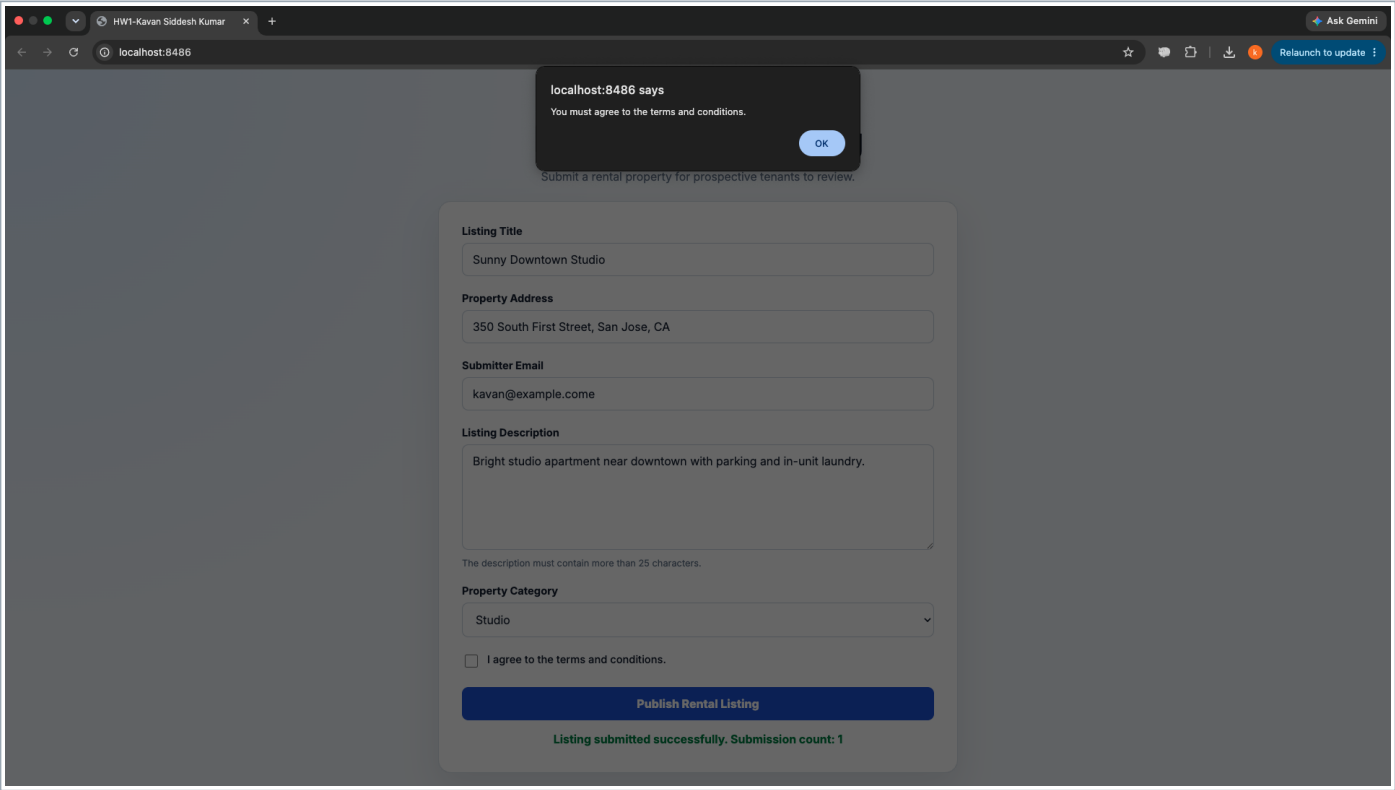


Figure 3. Terms-and-conditions validation alert on localhost:8486

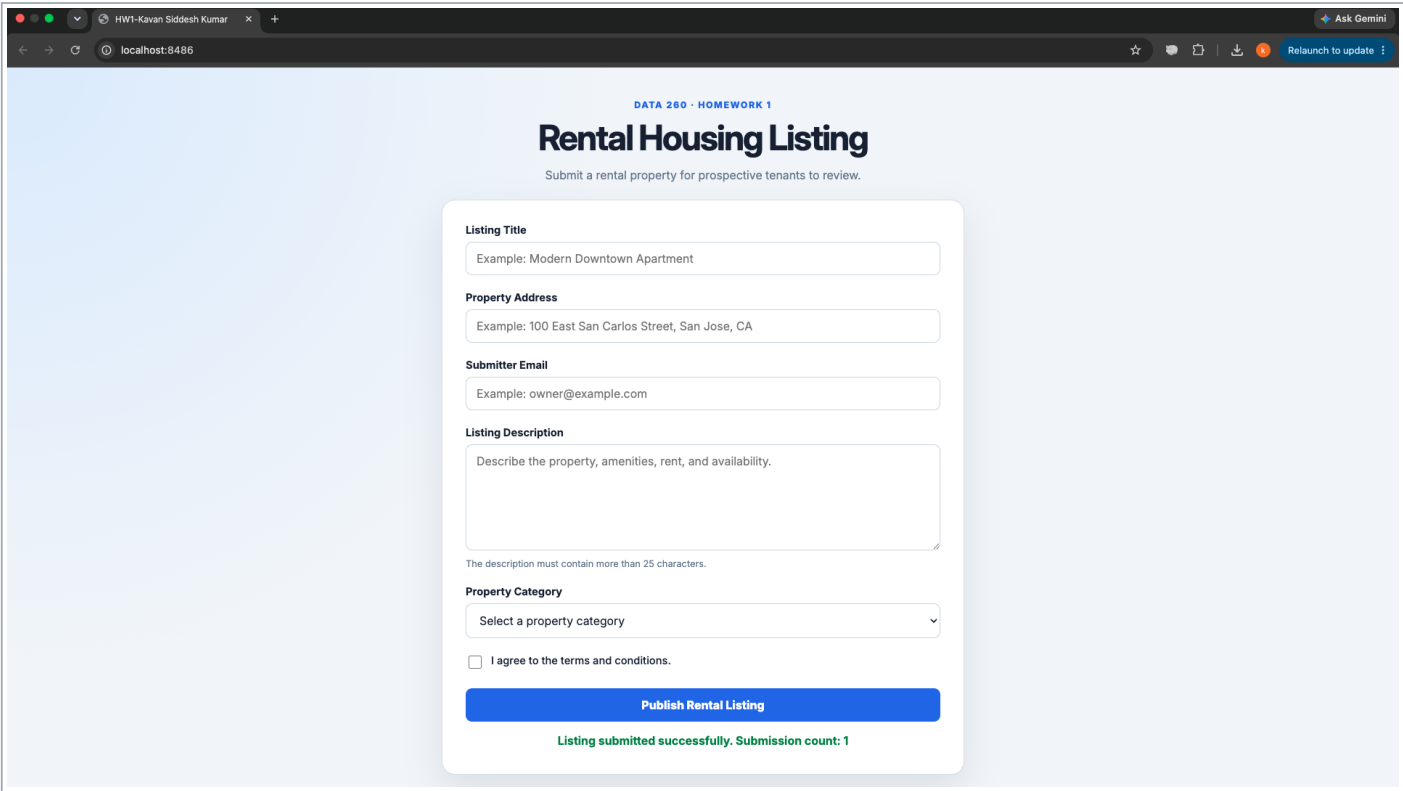


Figure 4. Successful rental-listing submission and closure count

Section 1. Part 1 - Docker Deployment

The same rental-listing application was built into a Docker image and served from container port 80 through local port 8486.

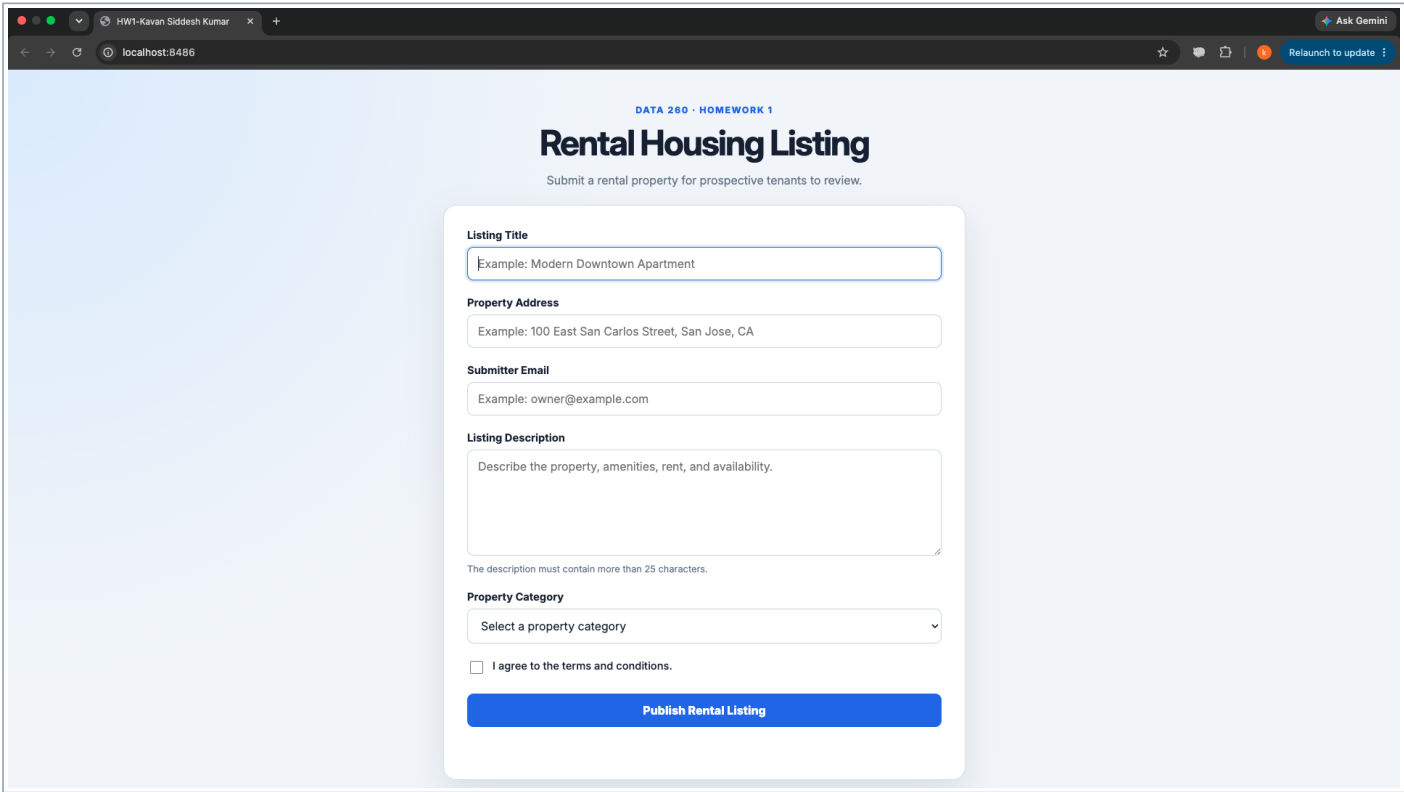


Figure 5. Rental application served by Docker on localhost:8486

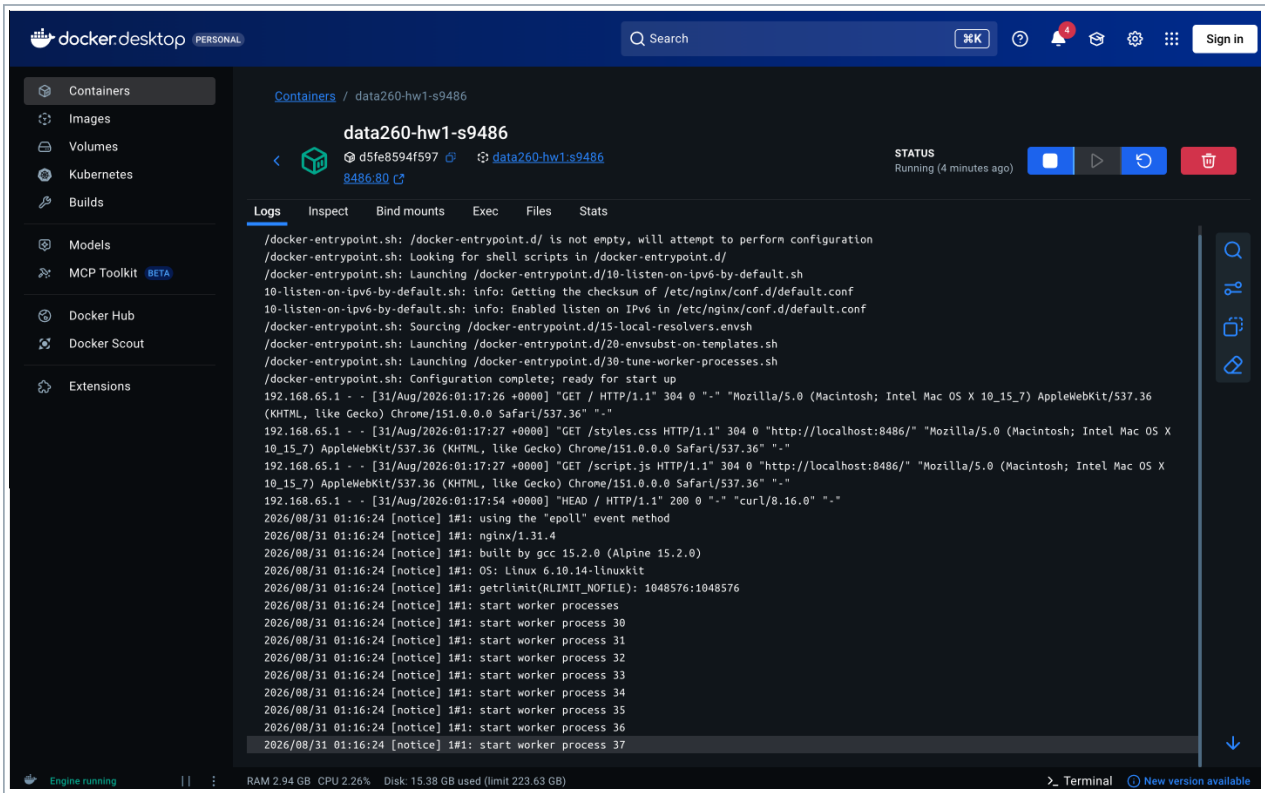


Figure 6. Docker Desktop container status and Nginx request logs

Section 1. Part 1 - AWS ECS Deployment

One Fargate task served the Docker image through a public IPv4 address. The paid resources were removed after verification.

The screenshot shows a web browser window with multiple tabs open. The active tab displays a web application titled "Rental Housing Listing" with the subtitle "Submit a rental property for prospective tenants to review." The browser's address bar shows the URL "18.232.31.124". The form contains the following fields and elements:

- Listing Title:** A text input field with the placeholder text "Example: Modern Downtown Apartment".
- Property Address:** A text input field with the placeholder text "Example: 100 East San Carlos Street, San Jose, CA".
- Submitter Email:** A text input field with the placeholder text "Example: owner@example.com".
- Listing Description:** A large text area with the placeholder text "Describe the property, amenities, rent, and availability." Below the text area, a note states "The description must contain more than 25 characters."
- Property Category:** A dropdown menu with the text "Select a property category".
- Agreement:** A checkbox labeled "I agree to the terms and conditions." which is currently unchecked.
- Submit Button:** A blue button labeled "Publish Rental Listing".
- Feedback Message:** A green message below the button stating "Listing submitted successfully. Submission count: 1".

Figure 7. Rental application running from the ECS task public IP

- Launch type: AWS Fargate; desired task count: 1.
- Task platform: Linux/X86_64; container port: 80.
- The browser address bar shows the public IPv4 endpoint used for verification.
- The ECS service, cluster, ECR repository, security group, and log group were removed after evidence was captured.

Section 2. Part 2 - Multi-Agent Rental-Domain Demo

Exact command

```
python agents_demo.py --title "Two-Bedroom Apartment Near SJSU" --content "A furnished two-bedroom apartment with in-unit laundry, secure parking, and convenient access to public transit is available near the San Jose State University campus." --email "kavan.siddeshkumar@sjsu.edu" --model qwen3:4b --strict | tee reports/hw01/raw/agent_demo_part2.txt
```

```
(data260) kavan@Kavans-MacBook-Air data260-9486 % python agents_demo.py \
--title "Two-Bedroom Apartment Near SJSU" \
--content "A furnished two-bedroom apartment with in-unit laundry, secure parking, and convenient access to public tra
nsit is available near the San Jose State University campus." \
--email "kavan.siddeshkumar@sjsu.edu" \
--model qwen3:4b \
--strict \
| tee reports/hw01/raw/agent_demo_part2.txt

--- Planner (38542 ms) ---
{
  "thought": "The task requires creating exactly 3 distinct, topical tags from the allowed candidates and a one-sentence
summary. The blog title is 'Two-Bedroom Apartment Near SJSU' and the content describes a furnished two-bedroom apartmen
t with specific features near San Jose State",
  "message": "Selected tags must be distinct, topical, and from the allowed list. Focusing on key features: 'furnished t
wo-bedroom apartment', 'secure parking', and 'convenient access to public transit' are the most relevant multi-word phra
ses that match the content and title.",
  "data": {
    "tags": [
      "secure parking",
      "two-bedroom apartment near sjsu",
      "furnished two-bedroom"
    ],
    "summary": "Furnished two-bedroom apartment near SJSU offers in-unit laundry, secure parking, and public transit acc
ess.",
    "issues": [
      "Tags must be exactly from allowed candidates"
    ]
  }
}

--- Reviewer (23259 ms) ---
{
  "thought": "The task requires 3 topical tags from allowed candidates and a concise summary. Key features in content: f
urnished, two-bedroom, near SJSU, secure parking, public transit. Must match exact tag candidates.",
  "message": "Selected tags: 'two-bedroom apartment near sjsu', 'furnished two-bedroom', 'secure parking' from allowed
list. Summary under 25 words.",
  "data": {
    "tags": [
      "two-bedroom apartment near sjsu",
      "furnished two-bedroom",
      "secure parking"
    ],
    "summary": "Furnished two-bedroom apartment near SJSU features in-unit laundry and secure parking with public transi
t access.",
    "issues": []
  }
}

Finalized Output
{
  "thought": "The task requires exactly 3 tags from allowed candidates and a concise summary. Reviewer confirmed tags ma
tch allowed list: 'two-bedroom apartment near sjsu', 'furnished two-bedroom', 'secure parking'. Summary is under 25 word
s and captures key features.",
  "message": "Tags selected from allowed candidates. Summary concise and within word limit.",
  "data": {
    "tags": [

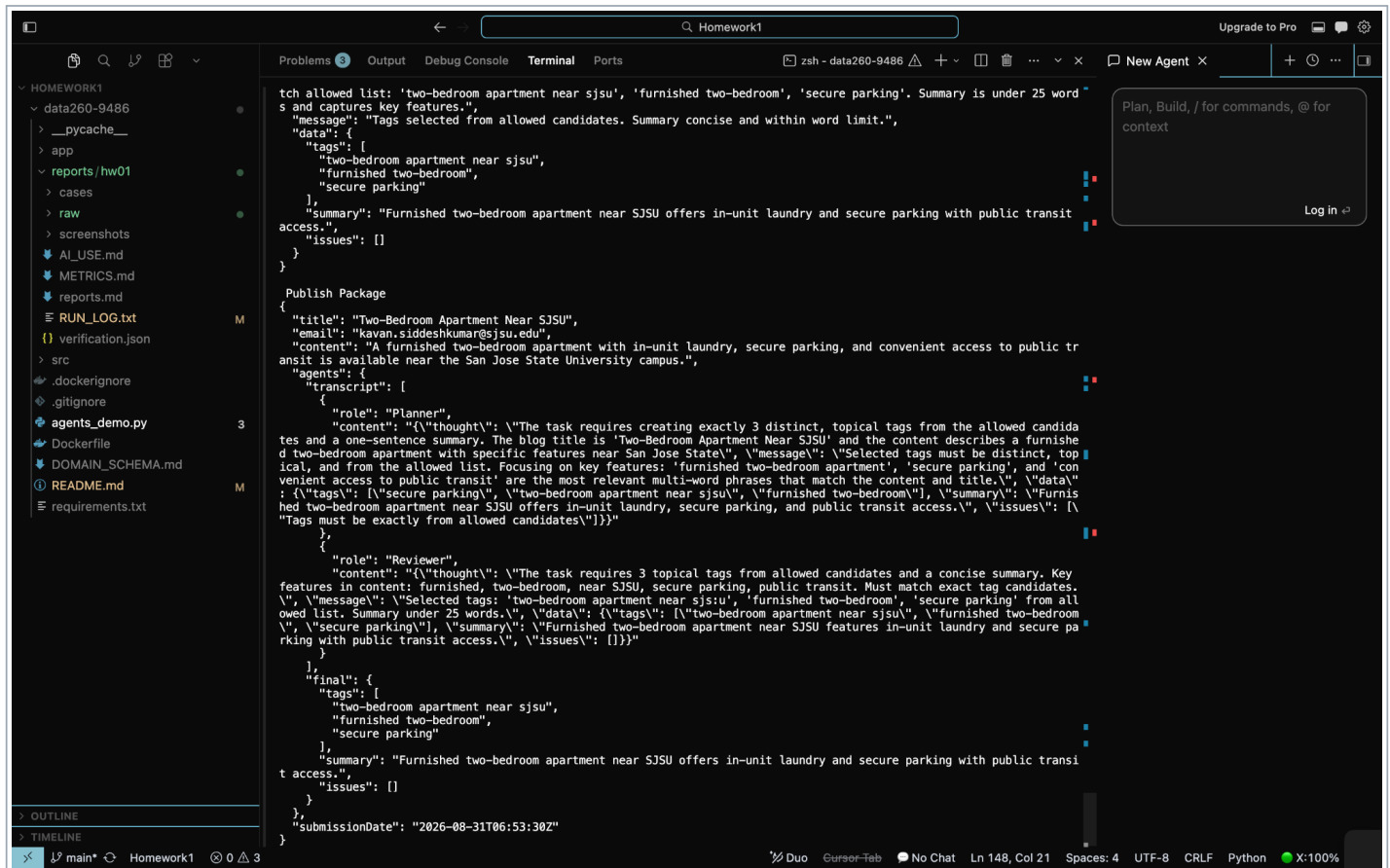
```

Figure 8. Planner and Reviewer outputs for the fixed rental-domain input

- Planner: proposes three topical candidates and a concise summary.
- Reviewer: checks relevance, word limits, and the required JSON structure.
- Finalizer: applies the review result and emits the normalized final package.

Section 2. Part 2 - Finalized Output

The final response contains exactly three tags, a summary under 25 words, an empty issues array, and the Publish Package JSON.



```

{
  "title": "Two-Bedroom Apartment Near SJSU",
  "email": "kavan.siddhikumar@sjsu.edu",
  "content": "A furnished two-bedroom apartment with in-unit laundry, secure parking, and convenient access to public transit is available near the San Jose State University campus.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "The task requires creating exactly 3 distinct, topical tags from the allowed candidates and a one-sentence summary. The blog title is 'Two-Bedroom Apartment Near SJSU' and the content describes a furnished two-bedroom apartment with specific features near San Jose State.",
        "message": "Selected tags must be distinct, topical, and from the allowed list. Focusing on key features: 'furnished two-bedroom apartment', 'secure parking', and 'convenient access to public transit' are the most relevant multi-word phrases that match the content and title.",
        "data": {
          "tags": [
            "secure parking",
            "two-bedroom apartment near sjsu",
            "furnished two-bedroom",
            "summary": "Furnished two-bedroom apartment near SJSU offers in-unit laundry, secure parking, and public transit access.",
            "issues": [ ]
          ]
        }
      },
      {
        "role": "Reviewer",
        "content": "The task requires 3 topical tags from allowed candidates and a concise summary. Key features in content: furnished, two-bedroom, near SJSU, secure parking, public transit. Must match exact tag candidates.",
        "message": "Selected tags: 'two-bedroom apartment near sjsu', 'furnished two-bedroom', 'secure parking' from allowed list. Summary under 25 words.",
        "data": {
          "tags": [
            "two-bedroom apartment near sjsu",
            "furnished two-bedroom",
            "secure parking"
          ],
          "summary": "Furnished two-bedroom apartment near SJSU features in-unit laundry and secure parking with public transit access.",
          "issues": [ ]
        }
      }
    ],
    "final": {
      "tags": [
        "two-bedroom apartment near sjsu",
        "furnished two-bedroom",
        "secure parking"
      ],
      "summary": "Furnished two-bedroom apartment near SJSU offers in-unit laundry and secure parking with public transit access.",
      "issues": [ ]
    }
  },
  "submissionDate": "2026-08-31T06:53:30Z"
}
```

Figure 9. Finalized Output and Publish Package

Required short answers

- Q1: two-bedroom apartment near sjsu; furnished two-bedroom; secure parking.
- Q2: Furnished two-bedroom apartment near SJSU offers in-unit laundry and secure parking with public transit access.
- Q3: No issues remained in the final output; data.issues was an empty array.

Section 3. Part 3 - Non-Determinism Measurement

The fixed input in reports/hw01/cases/nondeterminism_input.json was used unchanged for all 40 runs.

Metric	Temperature 0.0	Temperature 0.7
Runs	20	20
Distinct tag sets	1	3
Tags in all runs	furnished two-bedroom; secure parking; two-bedroom apartment near sjсу	furnished two-bedroom
Tags in exactly one run	None	convenient access
p50 latency	74523.73 ms	80115.68 ms
p95 latency	103968.01 ms	91286.90 ms
p99 latency	120482.96 ms	96006.23 ms

The screenshot shows a code editor with a file explorer on the left, a main editor window, and a terminal at the bottom. The file explorer shows a project structure with files like `data260-9486`, `reports/hw01`, `cases`, `raw`, `agent_demo_final.txt`, `agent_demo_part2.txt`, `agent_demo_strict.txt`, `nondeterminism_full_cons...`, `nondeterminism_metrics.j...`, `nondeterminism_runs.csv`, `nondeterminism_runs.json`, `nondeterminism_smoke_c...`, `screenshots`, `AI_USE.md`, `METRICS.md`, `reports.md`, `RUN_LOG.txt`, `verification.json`, `scripts`, `run_nondeterminism.py`, `src`, `.dockerignore`, `.gitignore`, `agents_demo.py`, `Dockerfile`, `DOMAIN_SCHEMA.md`, `README.md`, and `requirements.txt`. The main editor window shows the content of `nondeterminism_metrics.json`, which is a JSON object with two keys: `"0.0"` and `"0.7"`. The `"0.0"` key has a value object with `"runs": 20`, `"distinct_tag_sets": 1`, `"tags_in_all_runs": ["furnished two-bedroom", "secure parking", "two-bedroom apartment near sjсу"]`, `"tags_in_exactly_one_run": []`, and `"latency_ms": {"p50": 74523.73, "p95": 103968.01, "p99": 120482.96}}`. The `"0.7"` key has a value object with `"runs": 20`, `"distinct_tag_sets": 3`, `"tags_in_all_runs": ["furnished two-bedroom"]`, `"tags_in_exactly_one_run": ["convenient access"]`, and `"latency_ms": {"p50": 80115.68, "p95": 91286.9, "p99": 96006.23}}`. The terminal at the bottom shows the output of running the `run_nondeterminism.py` script, displaying the temperature, run number, completion time, and the tags discovered for each run. The output shows that at temperature 0.0, all runs discovered the same tags, while at temperature 0.7, different runs discovered different tag sets.

```
1 {
2   "0.0": {
3     "runs": 20,
4     "distinct_tag_sets": 1,
5     "tags_in_all_runs": [
6       "furnished two-bedroom",
7       "secure parking",
8       "two-bedroom apartment near sjсу"
9     ],
10    "tags_in_exactly_one_run": [],
11    "latency_ms": {
12      "p50": 74523.73,
13      "p95": 103968.01,
14      "p99": 120482.96
15    }
16  },
17  "0.7": {
18    "runs": 20,
19    "distinct_tag_sets": 3,
20    "tags_in_all_runs": [
21      "furnished two-bedroom"
22    ],
23    "tags_in_exactly_one_run": [
24      "convenient access"
25    ],
26    "latency_ms": {
27      "p50": 80115.68,
28      "p95": 91286.9,
29      "p99": 96006.23
30    }
31  }
32 }
```

```
Running temperature=0.7, run=12
Completed in 83634.64 ms: ['two-bedroom apartment near sjсу', 'furnished two-bedroom', 'secure parking']
Running temperature=0.7, run=13
Completed in 73240.58 ms: ['furnished two-bedroom', 'in-unit laundry', 'two-bedroom apartment near sjсу']
Running temperature=0.7, run=14
Completed in 76277.35 ms: ['two-bedroom apartment near sjсу', 'furnished two-bedroom', 'secure parking']
Running temperature=0.7, run=15
Completed in 97186.06 ms: ['two-bedroom apartment near sjсу', 'furnished two-bedroom', 'in-unit laundry']
Running temperature=0.7, run=16
Completed in 82134.68 ms: ['two-bedroom apartment near sjсу', 'furnished two-bedroom', 'secure parking']
```

Figure 10. Raw metrics JSON and recorded experiment runs

At temperature 0.0, identical requests produced one tag set. At 0.7, users could receive one of three tag sets. Variation is acceptable for optional discovery tags, but not for tenant eligibility, lease requirements, or legal-compliance decisions.

Section 4. Part 4 - Model Client and Token Accounting

hw1_client.py loads AGENT.md and sends every request through `src/model_client.ModelClient.complete(messages, tools=None)`.

Per-turn token counts

Turn	Input	Output	Total
1	125	256	381
2	407	256	663
3	699	256	955
4	986	256	1242
5	1272	256	1528

Recorded /stats snapshots

Snapshot	Turns	Cum. input	Cum. output	Cum. total	History length
After turn 3	3	1231	768	1999	4449 chars
After turn 5	5	3489	1280	4769	7102 chars

Required conceptual answers

- Prior context is resent because the chat API is stateless between requests.
- A system prompt sets global behavior; a user message supplies the current request.
- Input tokens grow because earlier messages are included again on every turn.
- Growth is limited by the model context window and practical latency and memory constraints.
- The recorded run shows the bullet-only checker reporting FAIL when the model used numbered lists, demonstrating that noncompliant output was detected honestly.

Section 4. Part 4 - Evidence After Turn 3

The `/stats` command reports counters without appending a new conversation message or changing history.

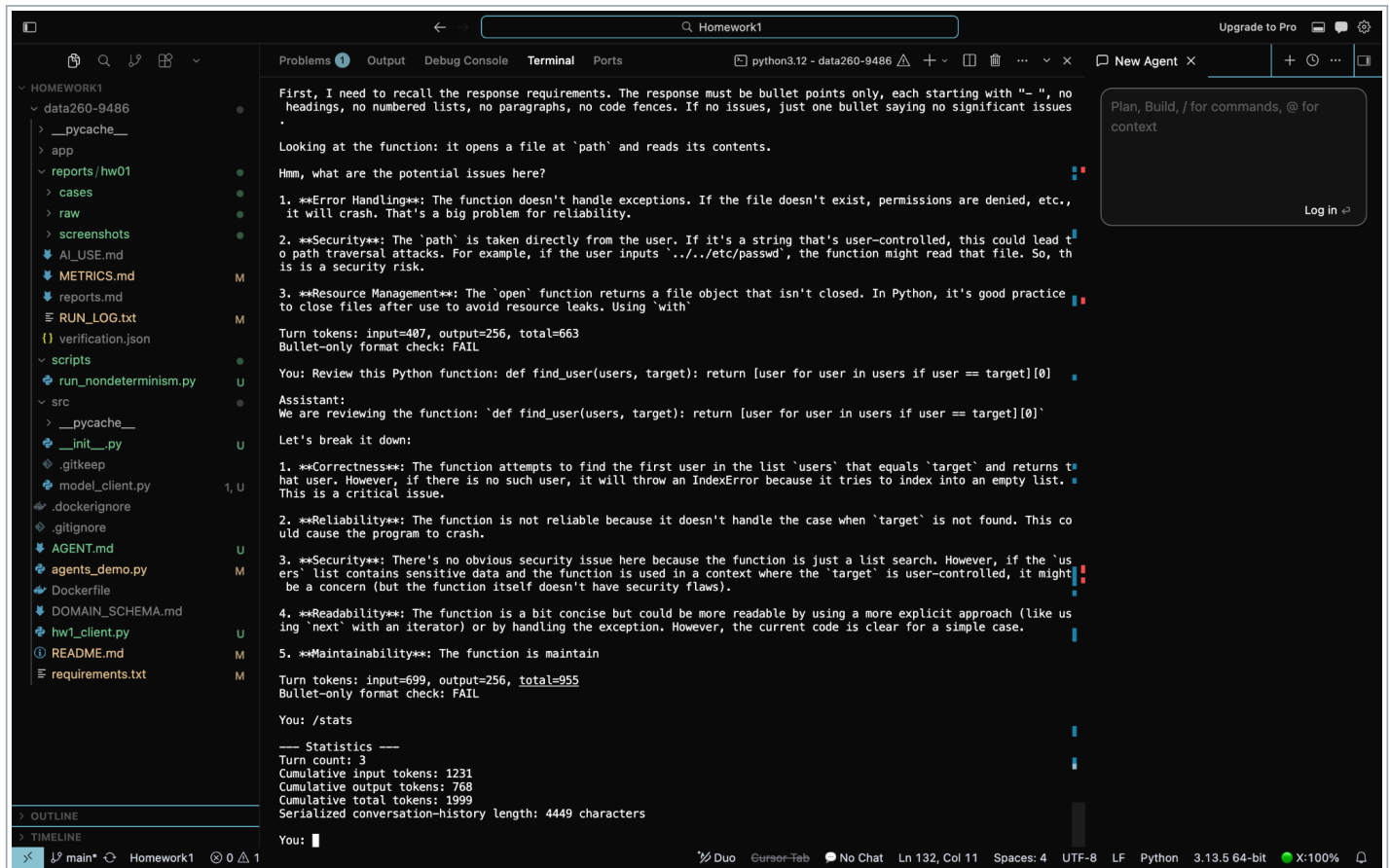


Figure 11. Turn 3 statistics and bullet-only format check

- Turn count: 3
- Cumulative input tokens: 1231
- Cumulative output tokens: 768
- Serialized conversation-history length: 4449 characters

Section 4. Part 4 - Evidence After Turn 5

The final snapshot demonstrates increasing input-token cost as conversation history grows.

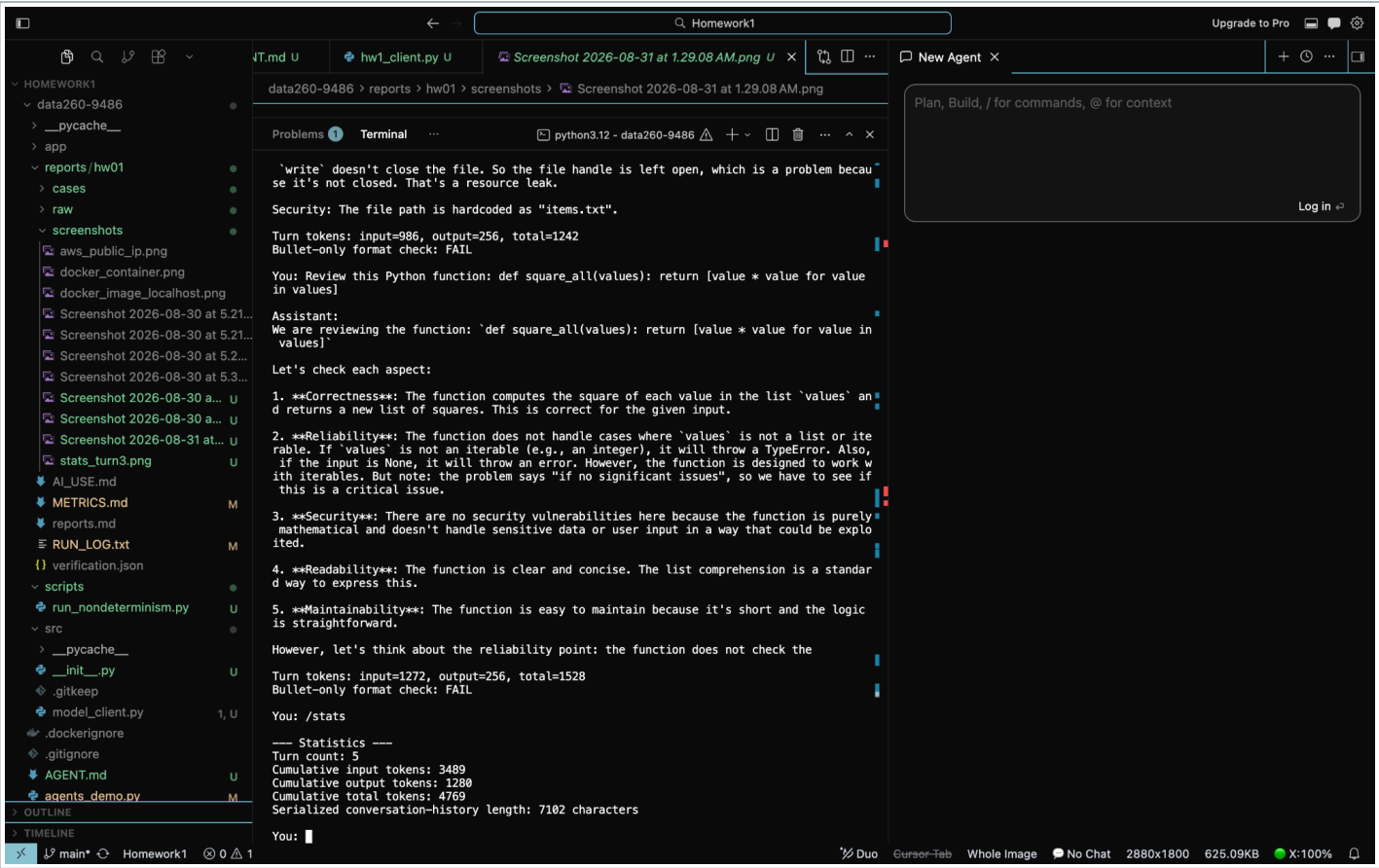


Figure 12. Turn 5 statistics and exit totals

Closing notes

- AI-use disclosure: reports/hw01/AI_USE.md
- Machine-readable verification: reports/hw01/verification.json
- Self-check script: scripts/verify_hw01.py
- Final submission artifact: reports/hw01/report.pdf